

Mesa de Servicios — Manual de Usuario

Version del plugin: 1.0.x · **Integrantes del equipo:** 10 · **Comandos:** 12

La Mesa de Servicios es un **equipo de asistentes de inteligencia artificial que recibe todo lo que llega de un cliente** — un ticket, un reporte de que algo falla, una cotización, un proyecto, un pedido de datos — desde cualquier fuente (un correo, un documento, la grabación de una reunión, un audio, un chat), y lo convierte en un **requerimiento cerrado**: sin ambigüedades, escrito en un documento que cualquiera entiende, y registrado en el tablero de tareas para que el equipo de desarrollo lo tome sin volver a preguntarle nada al cliente.

Tu hablas con el cliente; el equipo prepara las preguntas, las maquetas, los documentos y los tickets. No necesitas saber programar. Este manual está escrito para quien atiende la mesa, para quien dirige un área y para quien nunca abrió una herramienta técnica. La parte técnica (instalar y configurar) está al final, separada, en el **Anexo técnico**.

Índice

Parte 1 — Para todas las personas

1. [En dos minutos: como se usa](#)
2. [Conoce al equipo](#)
3. [Las palabras que vas a escuchar](#)
4. [Las reglas de la casa](#)
5. [Que puedes pedir \(lista de comandos\)](#)

Parte 2 — Situaciones reales, paso a paso

- [Caso 1: Me llegó un correo con un pedido](#)
- [Caso 2: Tuvimos una reunión con el cliente \(grabada o transcrita\)](#)
- [Caso 3: Cerrar las dudas con el cliente](#)
- [Caso 4: El cliente dice que algo falla](#)
- [Caso 5: El cliente pide algo nuevo con pantallas](#)
- [Caso 6: El cliente pide una cotización](#)
- [Caso 7: Hay que corregir o revertir datos](#)
- [Caso 8: Siempre hago lo mismo \(registrar un flujo propio\)](#)
- [Caso 9: Generar el documento y subir el ticket](#)
- [Caso 10: La memoria del negocio](#)

- [Caso 11: Ver como va la mesa](#)

Parte 3 — Preguntas frecuentes y problemas comunes

- [Preguntas frecuentes](#)
- [Si algo no anda](#)

Anexo tecnico (para quien instala y configura)

- [A. Instalacion y actualizacion](#)
 - [B. Configuracion de la mesa](#)
 - [C. La carpeta de la mesa](#)
 - [D. Cuidar el consumo](#)
 - [E. Solucion de problemas tecnicos](#)
-

Parte 1 — Para todas las personas

1. En dos minutos: como se usa

1. Alguien de tu area instala el plugin una vez (ver [Anexo A](#)).
2. Abres Claude Code **en la carpeta de la mesa** (la primera vez te ayuda a crearla).
3. Escribes esto y cuentas lo que llego, como se lo contarias a un colega:

```
/mesa-servicios:start
```

Ejemplos de lo que puedes escribir despues (o en la misma linea):

Lo que escribes	Lo que pasa
<code>/mesa-servicios:start</code>	El equipo mira como esta la mesa y te propone que hacer
<code>/mesa-servicios:start me llego un correo de ACME pidiendo un filtro nuevo</code>	Recibe el pedido y lo deja ordenado: que se entendio y que falta
<code>/mesa-servicios:start el cliente dice que se duplican las operaciones</code>	Lo reproduce en el sistema, con video y fotos, y confirma si pasa
<code>/mesa-servicios:start necesito cotizar una integracion con el banco</code>	Arma el requerimiento, la maqueta y la propuesta para el cliente
<code>/mesa-servicios:start ¿en que estamos?</code>	Te resume la mesa: que espera respuesta, que esta listo

Tres cosas que siempre van a pasar:

- **Nada se inventa.** Lo que el cliente no respondio queda escrito como pendiente, no como "lo que parece razonable".
- **Tu hablas con el cliente.** El equipo te prepara las preguntas, las maquetas y los documentos; nunca contacta al cliente por su cuenta.
- **Te presentan el plan antes de hacer nada** que cambie algo: antes de mandar preguntas, de maquetar, de armar scripts o de subir un ticket. Tu dices "adelante".

Si solo quieres saber algo del negocio ("¿que productos usa ACME?"), pregunta directo con `/mesa-servicios:brain query` . Es rapido y no mueve nada.

2. Conoce al equipo

Piensa en una mesa de atención al cliente con su propio analista, su probador, su encargado de datos, su maquettador y su redactor. Estos son sus 10 integrantes.

Integrante	Que hace, en palabras simples	Cuando aparece
 mesa-lead (jefe de mesa)	Decide que pedido sigue, reparte el trabajo, destraba y escala lo que no cierra. Es el único que puede pedir ayuda a los demás	Detrás de cada pedido; lo ves al aprobar planes
 recepcion (el que recibe)	Toma lo que llega (correo, documento, grabación, chat), lo guarda tal cual y escribe que se entendió, que falta y que se contradice	Cada vez que entra algo nuevo
 analista (el que pregunta)	Lee la memoria del negocio para no preguntar lo obvio y prepara pocas preguntas, claras, para que tu las lleves al cliente hasta que no quede duda	Siempre, hasta cerrar el pedido
 qa-negocio (el probador)	Reproduce lo que el cliente reporta en el sistema real, grabando todo (video, fotos, registro), y dice si pasa o no. Solo prueba; nunca busca la causa	Cuando el cliente dice que algo falla
 dba-mesa (el de los datos)	Mira los datos (solo mira) y arma los scripts para corregir o revertir, listos para que los ejecute quien corresponde. Nunca ejecuta nada	Pedidos de corregir, revertir o extraer datos
 prototipador (el maquettador)	Dibuja maquetas tipo boceto, navegables, para que el cliente vea la idea antes de que se construya	Pedidos nuevos con pantallas, proyectos, cotizaciones
 redactor (el que escribe el documento)	Convierte el pedido cerrado en un documento ejecutivo estándar (PDF y Word) y, si es cotización, en la propuesta para el cliente	Cuando el pedido está cerrado
 registrador (el del tablero)	Sube el ticket al tablero con todas sus etiquetas, el documento y las fotos dentro	Al final, para que desarrollo lo tome
 curador (el bibliotecario)	Guarda lo que se aprendió de cada pedido en la memoria del negocio, para que el próximo se cierre con menos preguntas	Al cerrar cada pedido
 setup (el que prepara la oficina)	Revisa que la computadora tenga todo e instala lo que falta, con tu permiso	Al empezar o cuando algo no está instalado

Sobre el costo: todos los integrantes usan el mismo "cerebro" intermedio por defecto. Si algun pedido es especialmente complejo, se puede subir a uno mas potente para un integrante (ver [Anexo B](#)).

3. Las palabras que vas a escuchar

Palabra	Que significa
Solicitud	Cualquier cosa que llega de un cliente y que la mesa va a trabajar: un pedido, un reporte, una cotizacion
Fuente	De donde salio la solicitud: un correo, un documento, la grabacion de una reunion, un chat. Se guarda tal cual, siempre
Ronda de preguntas	Un grupo corto de preguntas (como mucho siete) que el analista prepara y tu llevas al cliente. Normalmente bastan una o dos rondas; nunca mas de tres
Pedido cerrado	Cuando ya no queda nada por preguntar: se sabe que, para que, para quien, que incluye y que no
Tipo	De que se trata: BUG (algo falla), PRY (algo nuevo o un cambio), CTZ (algo que hay que cotizar y aprobar)
Alcance	Para quien es: un cliente especifico (ACME) o transversal (para todos los que usan el producto)
Correlativo	El numero de la solicitud: <code>BUG-2026-003</code> , <code>PRY-2026-014</code> , <code>CTZ-2026-007</code> . Unico, nunca se repite
Reproducir	Hacer en el sistema real, paso a paso, lo que el cliente dice que falla, para confirmarlo con evidencia
Evidencia	Las fotos, el video y los registros que demuestran lo que se vio. Se guardan y se muestran dentro del ticket
Maqueta (prototipo)	Un dibujo tipo boceto de las pantallas, que se puede recorrer, para validar la idea. No es el diseño final
Documento	El escrito ejecutivo estandar que cierra la solicitud. Sale en PDF y en Word
Propuesta	El documento para el cliente cuando hay que cotizar: que se hara, que no, plazos y condiciones. La valorizacion la pone comercial
Ticket	La tarjeta en el tablero (GitHub o Azure DevOps) con todo adentro: resumen, decisiones, pendientes, documento y fotos

Palabra	Que significa
Scripts	Instrucciones preparadas para corregir o revertir datos. La mesa los arma; los ejecuta quien esta autorizado
Flujo propio	Una operacion que la mesa repite (buscar operaciones de un cliente, revisar una bandeja) registrada paso a paso para hacerla siempre igual y con evidencia
faast-brain	La memoria del negocio: que productos hay, como opera cada cliente, que reglas rigen, que palabras usa cada uno. La mesa la lee antes de preguntar
Plan primero	La regla de que nadie manda preguntas, maqueta, arma scripts ni sube tickets sin mostrarte antes el plan y esperar tu "adelante"

4. Las reglas de la casa

Estan siempre activas. No hay que pedir las.

1. **Nada se inventa.** Lo que el cliente no respondio queda como pendiente, visible, nunca rellenado con suposiciones.
2. **Tu hablas con el cliente.** El equipo prepara; tu llevas y traes. Nadie del equipo contacta al cliente por su cuenta.
3. **Plan primero.** Antes de mandar preguntas, maquetar, armar scripts o subir un ticket, te muestran que se hara y esperan tu OK.
4. **Se pregunta poco y bien.** Primero se lee lo que el negocio ya sabe; se confirma lo sabido y se pregunta solo lo que falta, con opciones concretas, en el idioma del cliente.
5. **Todo en lenguaje de negocio.** Preguntas, documentos y tickets los entiende cualquier persona. Cero codigo, cero jerga.
6. **El probador solo prueba.** Reproduce, graba y reporta si pasa o no. Nunca busca la causa ni propone arreglos: eso es del equipo de desarrollo.
7. **El de los datos nunca ejecuta.** Mira los datos y arma los scripts, siempre seguros y repetibles. Los ejecuta quien esta autorizado, por el canal que corresponde.
8. **Toda evidencia se ve dentro del ticket.** Fotos y videos aparecen en la tarjeta, no como un enlace aparte.
9. **Una cotizacion no es trabajo** hasta que comercial la aprueba y queda registrado quien y cuando.
10. **Numero y alcance en todo.** Cada solicitud lleva su correlativo y dice si es de un cliente o transversal.
11. **Produccion, con permiso escrito.** Si un flujo propio o una reproduccion toca el ambiente productivo, se registra quien lo autorizo y para que, y el equipo se detiene antes de cada paso que cambie algo.

12. **Lo que se aprende se guarda.** Al cerrar cada solicitud, la memoria del negocio crece y la siguiente se cierra con menos preguntas.
13. **Sin secretos ni datos personales** en documentos, tickets ni memoria. Se habla de empresas y roles, no de personas.

5. Que puedes pedir (lista de comandos)

Recuerda: con `/mesa-servicios:start` y una frase basta. Esta lista es para cuando ya sabes exactamente que quieres.

Todos los días

Escribes	Que consigues
<code>/mesa-servicios:start</code>	El punto de partida: mira la mesa y te guía
<code>/mesa-servicios:estado</code>	Como va la mesa: que espera respuesta, que esta listo, que esta trabado
<code>/mesa-servicios:brain query {pregunta}</code>	Preguntarle a la memoria del negocio

Cuando llega algo

Escribes	Que consigues
<code>/mesa-servicios:recibir {archivos o texto}</code>	Recibe un pedido desde cualquier fuente (ver Caso 1 y Caso 2)
<code>/mesa-servicios:analizar {ID}</code>	Prepara la ronda de preguntas para el cliente, o registra sus respuestas (ver Caso 3)

Segun el tipo de pedido

Escribes	Que consigues
<code>/mesa-servicios:reproducir {ID}</code>	Reproduce lo que el cliente dice que falla, con video y fotos (ver Caso 4)
<code>/mesa-servicios:prototipo {ID}</code>	Maqueta tipo boceto para que el cliente vea la idea (ver Caso 5)
<code>/mesa-servicios:sql {que necesitas}</code>	Mira datos y arma scripts seguros, sin ejecutarlos (ver Caso 7)

Escribes	Que consigues
<code>/mesa-servicios:flujo {crear, grabar, ejecutar}</code>	Registra y repite tus flujos propios (ver Caso 8)

Para cerrar

Escribes	Que consigues
<code>/mesa-servicios:documento {ID}</code>	El documento ejecutivo en PDF y Word; la propuesta si es cotizacion (ver Caso 9)
<code>/mesa-servicios:ticket {ID}</code>	Sube el ticket al tablero con todo adentro
<code>/mesa-servicios:ticket aprobar {ID}</code>	Registra que comercial aprobo una cotizacion: recien ahi pasa a ser trabajo
<code>/mesa-servicios:brain ingest {ID}</code>	Guarda lo aprendido en la memoria del negocio (ver Caso 10)

Preparacion

Escribes	Que consigues
<code>/mesa-servicios:setup</code>	Revisa e instala lo que la computadora necesita, con tu permiso
<code>/mesa-servicios:brain init</code>	Crea la memoria del negocio por primera vez

Parte 2 — Situaciones reales, paso a paso

Cada caso muestra la situación, **que escribes exactamente**, que hace el equipo y que recibes al final. Los ejemplos son de un cliente ficticio ("ACME"), pero aplican a cualquiera.

Caso 1 — Me llegó un correo con un pedido

Situación: ACME manda un correo: "necesitamos que el listado de operaciones permita filtrar por rango de fechas y, de paso, el reporte semanal esta saliendo con totales raros".

Que escribes (pegando el correo o indicando el archivo):

```
/mesa-servicios:recibir correo-acme.eml --cliente ACME
```

Que hace el equipo:

1. El que recibe guarda el correo tal cual (nunca se modifica) y lo lee completo, incluido el hilo de respuestas.
2. Detecta que hay **dos pedidos de naturaleza distinta** en el mismo correo (un filtro nuevo y un reporte que falla) y te propone tratarlos como dos solicitudes.
3. Escribe el registro de cada una: que se entendió, los ejemplos concretos que dio el cliente, que falta, que se contradice entre el correo y respuestas anteriores, y si menciona un adjunto que no llegó.

Recibes:

```
SOL-2026-03-11-1 – Filtrar operaciones por rango de fechas – ACME – tipo preliminar: proyecto
```

```
Falta: ¿que perfiles lo usan? ¿fecha de operacion o de vencimiento? ¿solo pantalla o tambien exportacion?
```

```
SOL-2026-03-11-2 – Totales raros en el reporte semanal – ACME – tipo preliminar: bug
```

```
Falta: ¿que reporte exacto? ¿desde cuando? un ejemplo con un total que no cuadra  
Siguiente: /mesa-servicios:analizar SOL-2026-03-11-1
```

Caso 2 — Tuvimos una reunion con el cliente (grabada o transcrita)

Situacion: 40 minutos de reunion por Teams con ACME. Tienes la transcripcion que genero Teams, o solo la grabacion.

Que escribes:

```
/mesa-servicios:recibir reunion-acme-11-03.mp4 transcripcion-teams.docx --cliente ACME
```

Que hace el equipo:

- Si hay grabacion, la **transcribe en tu computadora** (nada sale a internet). Si hay transcripcion de Teams, la usa directo.
- Lee la reunion completa y separa lo que importa: **decisiones** (quien dijo que si a que), pedidos explicitos, compromisos de ambas partes, dudas que quedaron abiertas en la misma reunion, y sobre todo los **ejemplos concretos** que dio el cliente ("por ejemplo, cuando el proveedor manda la factura sin orden de compra...").
- Corrige los nombres de productos y terminos que la transcripcion automatica confunde, usando el vocabulario de la memoria del negocio.
- Separa lo que se pidio de lo que se descarto en la misma reunion.

Recibes: el registro de la solicitud con lo decidido, lo abierto y los ejemplos, listo para la ronda de preguntas.

Si hay un tramo inaudible en algo importante, no se rellena: queda como pregunta para el cliente.

Caso 3 — Cerrar las dudas con el cliente

Situacion: la solicitud esta recibida y faltan cosas por aclarar.

Que escribes:

```
/mesa-servicios:analizar SOL-2026-03-11-1
```

Que hace el analista:

1. Lee la memoria del negocio: que sabe ya de ACME, del producto y de casos parecidos. Lo que ya se sabe **no se pregunta**: se confirma en una frase ("entendemos que operan con el modo 2.0; si no es asi, avisanos").

2. Si hay ambiente disponible, mira la pantalla real de la que habla el cliente, para preguntar con contexto ("¿el total que falta es el de la tarjeta Resumen o el del listado?").
3. Prepara una **ronda corta** (como mucho siete preguntas), de lo mas importante a lo mas fino, con opciones concretas cuando las hay:

Ronda 1 – ACME – Filtro por fechas

Lo que ya tenemos claro (solo confirmar): el filtro va en la pantalla "Operaciones".

1. Fechas – ¿El rango filtra por fecha de operacion o de vencimiento? a) operacion b) vencimiento c) ambas d) otra: ____
2. Perfiles – ¿Quien lo usa? a) solo analistas b) analistas y supervisores c) todos
3. Exportacion – ¿El rango tambien debe aplicar al Excel que exportan? a) si b) no
- ...

4. **Te la muestra antes de "enviarla"** (plan primero). Tu la ajustas si quieres y la llevas al cliente por correo, chat o en la proxima reunion.

Cuando el cliente responde, se lo cuentas al equipo:

```
/mesa-servicios:analizar SOL-2026-03-11-1 --respuestas "1: b 2: b 3: si, pero solo el Excel resumido"
```

El analista registra las respuestas, marca lo que quedo decidido y lo que sigue pendiente, y revisa su lista de completitud. Si todo esta, **cierra** la solicitud y la clasifica: tipo, alcance (cliente o transversal), producto y prioridad sugerida. Si falta algo, prepara otra ronda (nunca mas de tres; si no cierra, te propone una reunion de trabajo).

Recibes:

SOL-2026-03-11-1 – CERRADA · Tipo: PRY · Alcance: Cliente ACME · Producto: Factoring · Prioridad sugerida: media
Necesita: maqueta si (cambia la pantalla) · reproduccion no · scripts no
Siguiente: /mesa-servicios:prototipo SOL-2026-03-11-1

Caso 4 — El cliente dice que algo falla

Situacion: "A veces se duplican las operaciones cuando paso a la pagina 2".

Que escribes (con los pasos ya cerrados con el cliente):

```
/mesa-servicios:reproducir SOL-2026-03-11-2
```

Que hace el probador:

1. Te muestra el plan: que pasos hara, en que ambiente, con que perfil, y que va a grabar. Espera tu OK. Por defecto usa el ambiente de pruebas; produccion solo si tu y el jefe de mesa lo autorizan, y queda registrado.
2. Hace **exactamente** los pasos del cliente, una vez, como un usuario. Graba todo: un video con las acciones marcadas y un capitulo por paso, una foto de cada paso con lo importante resaltado, y los registros internos.
3. Si se reproduce, lo repite una segunda vez para saber si pasa **siempre** o **a veces**.
4. Si algo lo frena antes de llegar al punto (no entra, el ambiente no responde), **se detiene y lo reporta**; no insiste ni busca atajos.

Recibes un veredicto claro:

```
Reproduccion SOL-2026-03-11-2 – Operaciones repetidas al cambiar de pagina
Veredicto: REPRODUCIDO (siempre) en pruebas · perfil analista
Pasos: 2 · Evidencia: 3 fotos, video 18 s, registros · Observacion: 1 llamada
interna con respuesta inesperada
Siguiente: /mesa-servicios:documento SOL-2026-03-11-2
```

Importante: el probador **nunca dice por que falla**. Solo demuestra que falla, con evidencia. La causa es del equipo de desarrollo. Si **no** se reproduce, no se inventa el bug: te dice que se probó, que si funciona, y que falta para volver a intentar (otro usuario, otro dato, otra hora).

Caso 5 — El cliente pide algo nuevo con pantallas

Situacion: el filtro por fechas cambia la pantalla de operaciones. Antes de escribir el documento, conviene que ACME **vea** la idea.

Que escribes:

```
/mesa-servicios:prototipo SOL-2026-03-11-1
```

Que hace el maquetaador:

1. Te propone las 3 a 6 pantallas clave (la de entrada, la del paso decisivo, la de resultado) y espera tu OK.
2. Arma una **maqueta tipo boceto**: cajas grises, dibujos de relleno, textos en las palabras del cliente, datos de ejemplo ficticios. Se abre en cualquier navegador y se recorre haciendo clic. A proposito **no parece terminada**: si pareciera terminada, el cliente la tomaria por terminada.
3. Saca una foto de cada pantalla y te prepara 3 a 5 preguntas de validacion por pantalla ("¿este es el orden en que trabajan?", "¿falta algun dato aqui?").

Tu se la muestras al cliente y le cuentas al equipo que decidio:

```
/mesa-servicios:prototipo SOL-2026-03-11-1 --validar "el rango va arriba del  
listado; agregar total de la seleccion"
```

Lo validado cierra preguntas y va al documento. Si cambia, se guarda una version nueva.

La maqueta aplica a pedidos nuevos que cambian pantallas, proyectos y cotizaciones. **No** se hace para errores (esos se documentan con evidencia) ni para pedidos sin cambio en pantalla. Tampoco es el diseño final: eso lo hace el estudio de diseño del equipo de desarrollo cuando el trabajo ya esta aprobado.

Caso 6 — El cliente pide una cotizacion

Situacion: ACME quiere integrar el sistema con su banco y necesita saber "cuanto y cuando".

Que pasa de distinto: una cotizacion es un proyecto que **todavia no esta aprobado**. Por eso:

1. Se cierra igual que un proyecto (Caso 3) y ademas se preguntan **entregables, supuestos, exclusiones, plazo esperado y quien decide**.
2. Se maqueta (Caso 5), porque una propuesta se entiende mucho mejor con pantallas.
3. El redactor genera **dos documentos**: el requerimiento interno y la **Propuesta para el cliente** (contexto, alcance, que se hara y que no, entregables, supuestos, plazos, condiciones y proximos pasos). La **valorizacion queda en blanco** para que la complete comercial; la mesa no pone precios ni esfuerzos.
4. El ticket se sube al tablero **en espera de aprobacion comercial**. No es trabajo todavia.

Cuando comercial aprueba:

```
/mesa-servicios:ticket aprobar CTZ-2026-007 "aprobado por Gerencia Comercial el 20/03/2026, propuesta v1"
```

Recien ahi la cotizacion pasa a la columna de trabajo y el equipo de desarrollo la puede tomar. Si se descarta, queda cerrada con el motivo (y lo aprendido se guarda igual).

Caso 7 — Hay que corregir o revertir datos

Situacion: "La operacion 10432 se giro por error; hay que revertirla".

Que escribes:

```
/mesa-servicios:sql script SOL-2026-03-12-4
```

Que hace el de los datos:

1. Con el pedido cerrado (que operacion exacta, en que estado esta y en cual debe quedar, por que, quien autoriza), **mira los datos:** confirma que la operacion existe, en que estado esta de verdad y que depende de ella. **Solo mira;** nunca cambia nada.
2. Te muestra el plan: que se tocara, cuantos registros, riesgos y quien lo ejecutara. Espera tu OK.
3. **Arma el paquete de scripts** con el formato que exige la casa (ordenado por motor, base y tipo de operacion), de forma que se pueda ejecutar dos veces sin daño, con su verificacion antes y despues y su reversa. Y un README en lenguaje claro: que hace, sobre que, riesgos y **quien debe ejecutarlo**.

Recibes: la carpeta del paquete y la indicacion de quien lo ejecuta. Si va a un ambiente formal, entra al pase del equipo de desarrollo, que lo revisa como cualquier otro.

Lo que nunca pasa: la mesa **no ejecuta** scripts contra ninguna base de datos. Ni en pruebas, ni "solo para validar". Arma, verifica y entrega; ejecuta quien esta autorizado.

Si el mismo pedido se repite (revertir operaciones es frecuente), se deja como **flujo reutilizable:** la proxima vez se rellena el numero de operacion y el paquete sale listo.

```
/mesa-servicios:sql flujo crear factoring/revertir-operacion  
/mesa-servicios:sql flujo preparar factoring/revertir-operacion SOL-2026-04-02-1
```

Caso 8 — Siempre hago lo mismo (registrar un flujo propio)

Situacion: cada semana buscas las operaciones vencidas de cada cliente en produccion y revisas una por una. Lo haces de memoria y cada persona lo hace distinto.

Registrarlo una vez:

```
/mesa-servicios:flujo crear factoring/buscar-operaciones-cliente
/mesa-servicios:flujo grabar factoring/buscar-operaciones-cliente
```

Te hace unas pocas preguntas (que hace, en que ambiente, con que perfil, si algun paso cambia algo) y luego **lo recorres tu con el equipo:** el probador captura cada paso con lo real (que pantalla, que boton, que se ve) y deja el instructivo escrito.

Repetirlo despues:

```
/mesa-servicios:flujo ejecutar factoring/buscar-operaciones-cliente cliente=ACME --
ambiente produccion
```

Te muestra el plan y, si es produccion, **registra quien lo autoriza y para que.** Luego verifica que esten dadas las condiciones (solo mirando), ejecuta los pasos grabando todo, y te entrega el resultado con la evidencia.

Dos clases de flujo:

- **Consulta** (solo mira: buscar, verificar, exportar): sin paradas, con evidencia de que solo se miro.
- **Accion** (algo cambia por la pantalla: aprobar, reasignar): el equipo **se detiene antes de cada paso que cambie algo** y espera tu OK en ese momento. En produccion, ademas, con la autorizacion registrada.

Dentro de un flujo, el de los datos solo **mira** (verifica antes y despues). Si el flujo necesita cambiar datos, eso es un paquete de scripts (Caso 7) para quien esta autorizado.

Caso 9 — Generar el documento y subir el ticket

El documento:

```
/mesa-servicios:documento SOL-2026-03-11-1
```

El redactor asigna el **correlativo** definitivo (`PRY-2026-014`) y el **alcance** (Cliente ACME o Transversal), y escribe el documento ejecutivo estandar: resumen para gerencia, que se pide y para que, quienes lo usan, que incluye y **que no**, el flujo deseado con los ejemplos del cliente, las pantallas de la maqueta, reglas, dependencias, criterios de exito y los **pendientes** explicitos. Sale en **PDF y Word**, con los acentos intactos. Si es cotizacion, tambien la Propuesta.

El ticket:

```
/mesa-servicios:ticket PRY-2026-014
```

El registrador te muestra titulo, etiquetas y columna, y con tu OK sube la tarjeta al tablero: titulo limpio, resumen, decisiones tomadas, pendientes, **fotos y video dentro** (no como enlace), el documento adjunto, y etiquetas completas (tipo, cliente o transversal, producto, prioridad sugerida, de donde vino). Si ya existia un ticket parecido, no duplica: te avisa.

Recibes:

```
PRY-2026-014 – Filtrar operaciones por rango de fechas · Cliente ACME  
Ticket #231 en el tablero · columna "Por priorizar" · 6 etiquetas · documento y  
maqueta adentro  
Siguiente: el lider de producto lo prioriza; /mesa-servicios:brain ingest PRY-2026-  
014
```

Desde aqui lo toma el equipo de desarrollo (plugin **dev-team**): lider de producto, historias de usuario, construccion, pruebas y publicacion.

Caso 10 — La memoria del negocio

La memoria (`faast-brain`) es lo que hace que la mesa **pregunte menos cada vez**. Guarda que productos hay y que hace cada uno, como opera cada cliente, que reglas rigen (plazos, topes, aprobaciones), que palabras usa cada uno y que se decidio antes.

Crearla la primera vez:

```
/mesa-servicios:brain init
```

Arranca casi vacia (con la lista de productos y clientes). Es normal: cada solicitud cerrada la alimenta.

Alimentarla al cerrar cada solicitud:

```
/mesa-servicios:brain ingest PRY-2026-014
```

El bibliotecario guarda **solo lo reutilizable**: una particularidad de ACME, una regla de negocio que nadie habia escrito, un termino nuevo, una decision, y un resumen del caso con "la pregunta que mas costo responder" (para hacerla antes la proxima vez).

Preguntarle:

```
/mesa-servicios:brain query ¿que productos usa ACME y como aprueban las operaciones?
```

Responde solo con lo que esta guardado, diciendo de donde salio. Si no lo sabe, lo dice; no inventa.

Lo que nunca entra: contraseñas, claves, ni datos personales de personas. Se habla de empresas y de roles. Si tienes la aplicacion gratuita **Obsidian**, puedes abrir la carpeta de la memoria y ver el mapa de como se conecta todo.

Caso 11 — Ver como va la mesa

```
/mesa-servicios:estado
```

Te muestra, en 20 segundos: cuantas solicitudes hay en cada estado, **que esta esperando al cliente y hace cuanto** (para recordarle), que esta trabado y por que, que cotizaciones esperan aprobacion, y que esta listo para el siguiente paso (documentar, subir ticket, guardar en la memoria). No cambia nada; solo mira y sugiere.

```
/mesa-servicios:estado --esperando      # solo lo que espera respuesta del cliente
/mesa-servicios:estado --cliente ACME    # todo lo de un cliente
```

Parte 3 — Preguntas frecuentes y problemas comunes

Preguntas frecuentes

¿Tengo que saber programar? No. Todo se pide y se recibe en lenguaje normal. Lo tecnico queda en el anexo.

¿El equipo le escribe al cliente? Nunca. Te prepara las preguntas, las maquetas y los documentos; tu los llevas y traes las respuestas.

¿Y si el cliente no responde? La solicitud queda "esperando cliente" con lo pendiente visible.
`/mesa-servicios:estado --esperando` te dice hace cuanto, para que le recuerdes. No se cierra a medias.

¿Cuántas rondas de preguntas puede haber? Como mucho tres. Si no cierra, el jefe de mesa te propone una reunion de trabajo o una decision de alcance.

¿Puedo recibir un audio de WhatsApp o un video? Si. Se transcribe en tu computadora; el archivo no sale a internet.

¿Por que el probador no me dice por que falla? Porque su valor esta en demostrar objetivamente que falla, con evidencia. La causa la busca el equipo de desarrollo.

¿El de los datos puede arreglar algo rapido en la base? No. Arma el script seguro y lo entrega; lo ejecuta quien esta autorizado. Asi nunca hay un cambio sin control.

¿Que pasa con una cotizacion? Se documenta, se propone y espera la aprobacion de comercial. Recien ahi es trabajo.

¿Como se numeran las solicitudes? TIPO-AÑO-NUMERO (BUG-2026-003). El numero es unico y nunca se reutiliza. Ademias, cada solicitud dice si es de un cliente o transversal.

¿Puedo usar la mesa con un tablero en Azure DevOps? Si. Se elige al configurar.

¿La mesa se puede usar junto con el equipo de desarrollo? Si. Usan carpetas distintas y la mesa deja el ticket justo donde el equipo de desarrollo empieza.

Si algo no anda

- **"No puedo registrar tickets"** → falta acceso al tablero: `/mesa-servicios:setup tracker` .
 - **"No transcribe el audio"** → falta el transcriptor: `/mesa-servicios:setup fuentes` .
 - **"No genera el PDF / Word"** → falta la herramienta de documentos: `/mesa-servicios:setup documentos` .
 - **"El probador dice que no puede abrir el sistema"** → revisa el ambiente y las cuentas de prueba: `/mesa-servicios:setup playwright` .
 - **"El de los datos no se conecta"** → revisa la conexión de solo lectura: `/mesa-servicios:setup db` .
 - **"Preguntas que el negocio ya sabía"** → la memoria está desactualizada: `/mesa-servicios:brain ingest --pendientes` .
-

Anexo tecnico (para quien instala y configura)

A. Instalacion y actualizacion

```
# 1. Agregar el marketplace (una sola vez)
/plugin marketplace add faast-app/faast-claude-marketplace

# 2. Instalar el plugin
/plugin install mesa-servicios@faast-marketplace

# 3. Actualizar cuando haya cambios publicados
claude plugin marketplace update faast-marketplace
claude plugin update mesa-servicios@faast-marketplace
# → reiniciar la sesion de Claude Code despues de actualizar
```

Requisitos (el agente `setup` los valida e instala con una confirmacion): `git` · `Node.js` ≥ 18 (navegador de pruebas; el servidor MCP de Playwright viene incluido) · `gh` (GitHub) o `az` (Azure DevOps) con acceso al repo y al Project · `ffmpeg` + Whisper (transcripcion local; el modelo se descarga la primera vez) · `pandoc` (lectura de documentos) · LibreOffice (PDF y Word) · clientes de base de datos `mysql` / `sqlcmd` **con cuentas de solo lectura**.

Si ya tenias un servidor MCP `playwright` registrado a mano en tu configuracion personal, quitalo para no duplicar herramientas: `claude mcp remove playwright`. El del plugin queda.

Modo nube (Claude en la nube o sesion remota): el `setup` verifica que existan los conectores necesarios (GitHub, correo/Teams si se usan) y advierte lo que no aplica (transcripcion de archivos grandes, bases en red privada).

B. Configuracion de la mesa

Todo vive en `.mesa/config.json` (lo crea `setup` la primera vez):

```

{
  "mesa": "mesa-faast",
  "elaboradoPor": "{Nombre Apellido}",
  "tracker": {
    "provider": "github",
    "github": { "org": "faast-app", "repo": "{repo-backlog}", "project": 12,
      "columnaInicial": "Por priorizar", "columnaCotizacion": "Pendiente
aprobacion comercial" }
  },
  "clientes": [ { "codigo": "ACME", "nombre": "ACME S.A.", "productos":
["factoring"] } ],
  "productos": ["factoring", "confirming", "leasing", "cobranza", "backoffice"],
  "brain": { "path": "~/Repositorios/faast-brain", "remote": "faast-app/faast-
brain" },
  "evidencia": { "repo": "faast-app/{repo-con-rama-evidence}" },
  "ambientes": { "pruebas": { "url": "https://qa...", "cuentasEnv":
["MESA_QA_ANALISTA"] },
    "produccion": { "url": "https://...", "requiereAutorizacion": true
  } },
  "fuentes": { "transcripcion": { "motor": "whisper", "modelo": "medium", "idioma":
"es" }, "conectores": [] },
  "team": { "models": { "analista": "opus" } }
}

```

Campo	Efecto
<code>tracker.provider</code>	<code>github</code> / <code>azure</code> : donde se registran los tickets
<code>tracker.github.project</code> / <code>columnaInicial</code> / <code>columnaCotizacion</code>	El Project y las columnas destino (bugs y proyectos a una; cotizaciones a la de espera)
<code>clientes[]</code>	Codigo corto, nombre y productos de cada cliente (para el alcance y las etiquetas)
<code>productos[]</code>	Catalogo de productos (etiquetas y memoria)
<code>brain.path</code> / <code>brain.remote</code>	Donde vive la memoria del negocio (<code>faast-brain</code>)
<code>evidencia.repo</code>	Repo donde se publican fotos y documentos para verlos dentro del ticket (rama <code>evidence</code>)

Campo	Efecto
<code>ambientes.*</code>	URLs y nombres de las variables con las cuentas de prueba; produccion exige autorizacion
<code>fuentes.transcripcion</code>	Motor y modelo de transcripcion local
<code>team.models.{agente}</code>	Subir un integrante a un modelo mas potente solo en esta mesa (default <code>sonnet</code>)

Secretos que nunca entran a git: `.mesa/accesos.env` (cuentas de prueba), `.mesa/db-access.json` (conexiones de solo lectura), `.mesa/setup-status.json` , toda la carpeta `.mesa/evidencia/` , las carpetas `evidencia/` de cada solicitud y los archivos de audio/video de las fuentes.

C. La carpeta de la mesa

`.mesa/` es la memoria operativa de la mesa. Es independiente de `.coordination/` del plugin dev-team: pueden convivir.

```
.mesa/
├─ config.json          configuracion (arriba)
├─ correlativos.json    ultimo numero por tipo y año (BUG/PRY/CTZ)
├─ backlog.md           espejo: una linea por solicitud con su ticket y estado
├─ solicitudes/{ID}/    fuentes/ (originales) · solicitud.md · preguntas.md ·
estado.json            |
                        |          prototipo/ · evidencia/ (gitignored) · scripts/ ·
documento/             |
├─ flujos/{dominio}/{flujo}/    flujos propios de la mesa (se versionan)
├─ flujos-sql/{dominio}/{flujo}/ flujos de scripts reutilizables (regla global de
formato)
├─ evidencia/           corridas de flujos y staging del navegador – gitignored
├─ handoffs/           notas entre integrantes
├─ metrics/            activity.jsonl (actividad automatica)
├─ accesos.env         cuentas de prueba – NUNCA en git
├─ db-access.json      conexiones de solo lectura – NUNCA en git
└─ setup-status.json   estado del entorno – NUNCA en git
```

Evidencia en el ticket: en GitHub se publica en la rama huerfana `evidence` del repo configurado (`evidence/mesa/{ID}/`) y se embebe en el issue; en Azure DevOps, como adjunto embebido en el item. Nunca en una rama de codigo.

Scripts del DBA de mesa: el paquete ejecutable va estrictamente en `MOTOR/N_base/N_tipo.sql` (por ejemplo `MYSQL/1_db_fintec/7_update.sql`); las verificaciones `00-*.sql` (solo lectura) quedan fuera y la reversa en `reversa/` con el mismo formato. La mesa nunca ejecuta.

D. Cuidar el consumo

- Un pedido chico no necesita a todos: el jefe de mesa decide quien participa (un bug sin pantalla no se maqueta; un pedido sin datos no llama al de los datos).
- Las rondas cortas y la memoria del negocio ahorran mas que cualquier otra cosa: cada pregunta que el negocio ya responde es una ronda menos.
- Todos los integrantes corren en el modelo intermedio por defecto; sube solo al que lo necesite (`team.models`).
- `/mesa-servicios:estado` y `/mesa-servicios:brain query` son de solo lectura: usalos sin miedo.

E. Solucion de problemas tecnicos

Sintoma	Causa probable	Que hacer
No aparecen las herramientas <code>browser_*</code>	Sesion vieja o MCP duplicado	Reiniciar Claude Code; <code>claude mcp list</code> debe mostrar un solo <code>playwright</code>
<code>gh</code> dice que no tiene acceso al Project	Falta el permiso <code>project</code>	<code>gh auth refresh -s project,read:project</code>
Whisper tarda mucho la primera vez	Descarga del modelo	Normal; esperar. Para equipos modestos usar <code>faster-whisper</code> o el modelo <code>small</code>
El PDF sale sin acentos	Conversion con fuente incorrecta	<code>/mesa-servicios:setup documentos</code> (prueba real de conversion)
La cuenta de base de datos puede escribir	Cuenta mal creada	Pedir una de solo lectura; el setup lo reporta como riesgo y no la usa para escribir
Dos solicitudes con el mismo numero	Edicion manual de <code>correlativos.json</code>	No editar a mano; el redactor lo lleva de forma atomica